FoodGPT: A Rule-Grounded, LLM-Assisted Nutrition and Meal-Planning Dashboard for Bangladeshi and International Cuisine

Abstract - Public food-composition datasets are accurate but hard for a non-specialist to use: knowing that a CSV has a sodium column does not answer "is this safe for me to eat" or "what should I cook tonight." This report presents FoodGPT, a Streamlit dashboard built around one cleaned Bangladeshi and international food-nutrition dataset (1,254 dishes, 22 columns) that is shared by eight purpose-built tools - nutrition lookup, health-condition recommendations, recipe generation, calorie-targeted meal planning, free-text menu matching, allergen and diet filtering, side-by-side food comparison, and a conversational assistant - so that a number shown in one tool always matches the same number in another. Every tool is deterministic and rule- or heuristic-driven except the chat assistant, which first tries five regex intents against the dataset and, only for open-ended questions, falls back to a Groq-hosted large language model that is grounded with dataset rows retrieved for that message and degrades to a dataset-only reply if the model call fails. We describe the dataset-cleaning and allergen-estimation pipeline, the portion-scaling, health-scoring and meal-planning heuristics, the two-stage conversational pipeline, and the overall system architecture, and we discuss the reliability and scalability trade-offs of choosing transparent rules over learned models for a safety-sensitive, single-developer-maintainable course project.

Index Terms - *nutrition informatics, food recommender systems, retrieval-augmented generation, allergen detection, diet planning, conversational agents, Streamlit.*

I. INTRODUCTION

Nutrition data is abundant but rarely actionable. A raw spreadsheet of calories, macronutrients and dietary flags answers a query only if the reader already knows how to combine those numbers with a health goal, an allergy, or a recipe. FoodGPT was built to close that gap for three everyday questions that a household in Bangladesh - or anywhere - asks about food: is this dish safe for me to eat given a condition or allergy, what should I cook tonight with the ingredients or goals I have, and what does a full day of meals look like for a given calorie target. Rather than answering these from a language model's general knowledge, which risks fabricating nutrition numbers that matter for conditions such as diabetes or hypertension, every answer in FoodGPT is traced back to one cleaned dataset that every tool in the application shares.

Bangladeshi cuisine in particular is underserved by mainstream nutrition apps, most of which are built around Western or generically international food databases; a dish such as Rui Bhuna or Khichuri either has no entry at all or is mapped to a poor international proxy, which silently degrades any downstream calorie or allergen calculation. Building FoodGPT around a Bangladeshi-and-international dataset (Section III) is therefore not just a data-source choice but a design goal: the eight tools and the chat assistant should give a Bangladeshi household the same quality of answer for a home-cooked fish curry that a Western nutrition app gives for a sandwich.

The project also serves as a case study in when to reach for a large language model and when not to. Of the eight tools described in this report, seven are entirely rule-based or heuristic: they look up, filter, score, and scale rows of a pandas DataFrame with no learned model in the loop. Only the eighth, the Chat Assistant, calls an LLM, and even then only after a bank of regex intents has failed to answer the question deterministically, and only with the model's context restricted to dataset rows retrieved for that specific message. This design keeps the parts of the application that most affect user safety - allergen and condition matching - fully deterministic and auditable, while still offering the open-ended, conversational experience that a plain form-based UI cannot.

The contributions of this project are:

1) A reusable, single-source dataset-cleaning and allergen-estimation module (data.py) that every other module in the codebase imports, so no parsing or allergen logic is duplicated across the eight tools;

2) Eight task-specific Streamlit tools that read one cached, cleaned DataFrame, guaranteeing that the same food shows identical numbers in every tool;

3) A two-stage conversational assistant that answers deterministically from the dataset whenever a rule matches, and otherwise falls back to a retrieval-grounded LLM call with a graceful, dataset-only degradation path when that call is unavailable or fails; and

4) Portion-scaling, meal-planning and recipe-generation heuristics that size any dataset food or recipe to an arbitrary target weight or a calorie budget split across meals, without requiring a trained model or external nutrition API.

II. RELATED WORK

Food and nutrition recommendation has an established literature. Toledo et al. [1] propose a hybrid content- and preference-based food recommender that balances nutritional suitability against a learned user-preference model; FoodGPT's Health Recommendations tool addresses a similar goal— matching foods to a condition or objective - but with hand-authored, transparent threshold rules (Section V-C) rather than a trained preference model, trading personalization for auditability and for not requiring a ratings history that a course-scale dataset does not have. Yera Toledo et al. [2] extend this line of work with a health-awareness recommender that filters and ranks foods for conditions such as diabetes and hypertension; FoodGPT's condition-fit rules cover an overlapping set of conditions (diabetes, hypertension, heart disease, weight loss, weight gain) using simple boolean and numeric-threshold logic on dataset flags, which keeps every recommendation explainable in one sentence.

A second strand of work recognizes food from images rather than text. Ocay and Fernandez [3] describe NutriTrack, an Android app that classifies a photographed dish and reports its nutrition, and Han et al. [4] describe NutrifyAI, which combines real-time food detection with LLM-generated meal recommendations. FoodGPT deliberately avoids computer vision: because the target dataset already enumerates 1,254 named Bangladeshi and international dishes, exact and fuzzy text matching (Section V-F) is sufficient to resolve a typed food name, a menu line, or a chat message to a dataset row, which removes an entire class of vision-model failure modes at the cost of requiring the food to already exist in the dataset or to be spelled recognizably close to an entry.

FoodGPT's Chat Assistant follows the retrieval-augmented generation (RAG) paradigm introduced by Lewis et al. [5], in which a language model's context is populated with passages retrieved from an external

corpus rather than relying solely on parameters learned at training time, which has been shown broadly to reduce hallucination in knowledge-intensive tasks [5], [6]. Where a typical RAG system indexes a document corpus with a dense vector retriever, FoodGPT's retrieval step (Section VI) is a lightweight keyword-overlap score over the same cleaned DataFrame that every other tool queries; this keeps the chat assistant's source of truth identical to the rest of the application rather than introducing a second, separately maintained knowledge base.

Finally, the allergen-estimation logic in data.py sits within the broader field of food-domain text mining, surveyed by Xiong et al. [7], and is related to rule- and NLP-based allergen-flagging systems such as the OCR- and NLP-based allergen notifier of [8]. Consistent with the survey's observation that rule-based extraction remains competitive with learned NER models when the label vocabulary is small and precision on "contains X" claims is safety-critical, FoodGPT uses hand-curated keyword lists with a two-tier "contains / may contain" confidence level (Section III) rather than a trained named-entity recognizer, which would need labeled training data this project does not have.

FoodGPT's Chat Assistant is also usefully contrasted with a general-purpose conversational model used without any grounding, such as a person simply asking a consumer chatbot "is Rui Bhuna okay for diabetes." A general-purpose model has almost certainly not been trained on this specific dataset's numbers and would either decline to answer precisely or, more concerningly, generate a plausible-sounding sugar or sodium figure that is not actually this dataset's value. By construction, Stage 1 of FoodGPT's assistant (Section VI-A) answers that exact question deterministically from the same row every other tool in the application uses, and Stage 2 (Section VI-B) restricts the model to reasoning only over rows retrieved from that row set, so the assistant's worst case for an in-dataset food is a poorly phrased but numerically grounded answer, not a fabricated one.

Across all four strands of related work, a common design tension is precision versus coverage: learned models (a preference ranker, a vision classifier, a dense retriever, a trained NER tagger) generalize further than hand-written rules but require training data, are harder to audit for a specific wrong answer, and can fail silently. FoodGPT's guiding choice throughout Sections III, V and VI is to accept the coverage cost of rules and keyword lists - a food or an allergen phrasing outside the authored vocabulary is simply missed - in exchange for the ability to point at the exact line of code responsible for any single output, which the authors judge to be the right trade-off for a project whose outputs can influence what someone with diabetes or a food allergy chooses to eat.

III. DATASET AND PREPROCESSING

Data provenance and licensing: The final submission should identify the original source(s) of bd_food_nutrition_dataset.csv, its license/usage permission, the date accessed, and any preprocessing or manually authored rows. If the dataset was assembled by the project team, state the collection and validation procedure explicitly.

The dataset, bd_food_nutrition_dataset.csv, holds 1,254 foods across 22 columns, summarized in Table I. Foods span 22 categories, the largest being Fish (363 foods), Vegetable (224), Meat (144), Fruit (60) and Rice Dish (57), shown in Fig. 3; 906 rows are tagged purely Bangladeshi cuisine with the remainder split across international and fusion labels; and dishes are distributed across Dinner (482), Lunch (475), Snack (187), Dessert (55), Breakfast (40) and Starter (15) meal types. The mean listed serving carries 213 kcal and 13.3 g of protein, reflecting a dataset weighted toward normal meal portions rather than whole-day totals.

Field	Description
food_id	Unique integer identifier for each of the 1,254 dishes
food_name	Dish name, e.g. "Rui Bhuna"
category	One of 22 categories (Fish, Vegetable, Meat, ...)
cuisine	Bangladeshi, International, or a fusion label
meal_type	Breakfast, Lunch, Dinner, Snack, Dessert or Starter
serving_size_g	Listed reference serving weight, in grams
7 nutrient cols.	calories_kcal, protein_g, carbs_g, fat_g, fiber_g, sugar_g, sodium_mg per listed serving
8 diet flags	vegetarian, vegan, gluten_free, diabetic_friendly, high_protein, low_fat, heart_healthy, weight_loss_friendly
ingredients, tags	Free-text ingredient list and searchable keyword tags

TABLE I. SCHEMA OF THE 1,254-ROW FOOD NUTRITION DATASET.

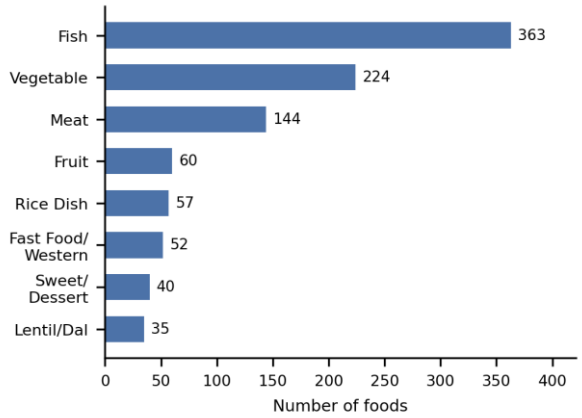

Fig. 3. Distribution of the eight largest food categories in the dataset. (Insert/retain the actual chart image here.)

All access to the raw CSV is centralized in support/core/data.py so that no other module parses the file directly. load_food_dataset() locates the CSV (or an uploaded replacement of the same shape) and clean_food_dataset() normalizes column names, resolves several accepted spellings per field (for example calories, calories_kcal, kcal and energy all resolve to one calories column) so that a differently formatted dataset can be dropped in without code changes, coerces every nutrient to a number with missing values filled as zero, and coerces the eight dietary-flag columns from Yes/No text to booleans. The cleaned frame is cached with Streamlit's st.cache_data so the roughly 1,254-row cleaning pass runs once per session rather than on every page load.

A. Allergen Estimation

Because the dataset does not ship allergen labels, detect_allergens() estimates ten common allergens (dairy, egg, fish, shellfish, gluten, peanut, tree nuts, soy, sesame, mustard) per food from its name, ingredient list, tags and category. For each allergen, two word-boundary regular expressions are compiled from curated keyword lists: a "sure" list (for example "milk", "cheese", "paneer" for dairy) that marks the food as "contains", and a broader "maybe" list (for example "cake", "naan", "halwa") that marks it "may contain". Category-level priors fill gaps where the text gives no lexical cue - every Bakery item defaults to "may contain" dairy, egg and gluten even if the word "flour" never appears - and the dataset's own gluten_free, vegetarian and vegan flags are used to both add gluten as "contains" when the flag is false and to retract spurious "may contain" dairy, egg, fish or shellfish flags on foods already marked vegan or vegetarian. The result is stored per row as a small dictionary and rendered as colour-coded

pills throughout the UI (Fig. 1), so every screen that lists a food also shows why it might be unsafe for a given allergy.

B. Diet Filters

Twelve named diet filters (vegetarian, vegan, pescatarian, eggless, dairy-free, gluten-free, nut-free, no-beef, no-pork/halal-friendly, no-red-meat, low-sodium, low-sugar) are implemented as small rule functions in diet_violations(), reusing the allergen dictionary above plus three additional keyword patterns for beef, pork/alcohol, and red meat in general, and two numeric thresholds:

$$sodium\_mg \le 400 \ \ (low\ sodium) \quad | \quad sugar\_g \le 8 \ \ (low\ sugar) \tag{1}$$

food_filter_report() unions every violated diet rule, every requested allergen at the appropriate confidence level, and any user-supplied avoid-words into one reasons list per food; a food passes only when that list is empty, so the Diet & Allergen Filter tool (Section V-G) can show the visitor exactly why any hidden food was hidden rather than silently dropping it.

IV. SYSTEM ARCHITECTURE

FoodGPT is organized in three layers (Fig. 1). The presentation layer is a nine-page Streamlit application - a landing Dashboard plus eight feature pages - registered from a single FEATURES list in support/core/theme.py, so that adding a new tool to the top navigation and the dashboard hero grid requires one new entry rather than touching every page. The core-logic layer, support/core/, holds four modules: data.py for loading, cleaning and allergen/diet-filter logic; nutrition.py for scoring, portion scaling, recipe text and meal planning; chatbot.py for the two-stage conversational pipeline; and theme.py for the shared dark-mode styling and navigation chrome used by all nine pages. The data layer is the single cached, cleaned DataFrame produced by get_prepared_dataset(); every page and every core function reads from this one object, which is the central design decision behind the ““one number, everywhere”” guarantee described in Section I. A fourth, optional layer - the Groq-hosted large language model - is reached only from chatbot.py and only when the rule-based stage does not resolve a chat message, keeping the application's seven other tools fully independent of any external network call or API key.

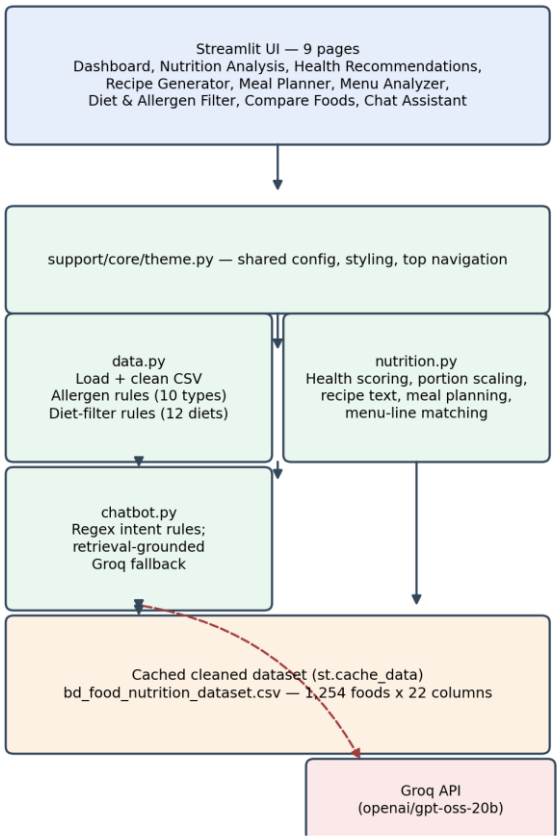

Fig. 1. Three-layer system architecture: Streamlit UI, shared core-logic modules, and the single cached dataset that every tool and the chat assistant read from.

The Dashboard page is the application's entry point rather than a ninth independent tool: it renders one hero grid of tiles, one per entry in the FEATURES registry, and delegates entirely to st.page_link() for navigation, so it holds no logic of its own beyond what theme.py already provides to every other page. This means the registry in theme.py is, in effect, the application's routing table, and the architectural claim that “adding a tool costs one registry entry” (Fig. 1) applies equally to the Dashboard's own hero grid and to the top navigation bar shown on the other eight pages.

V. METHODOLOGY

A. Portion Scaling and Daily-Value Context

The Nutrition Analysis tool rescales any food's listed nutrition to an arbitrary serving weight. scale_row() computes a scale factor from the requested grams and the dataset's own base serving size, and applies it to all seven tracked nutrients:

$$scaled\_n = base\_n \times (grams / base\_serving\_g) \tag{2}$$

and expresses each scaled value against a fixed 2000 kcal / 50 g protein / 275 g carbs / 70 g fat / 28 g fiber / 50 g sugar / 2300 mg sodium adult reference day as a percentage:

$$DV\%\_n = (scaled\_n / reference\_n) \times 100 \tag{3}$$

portion_advice() then turns the scaled values into plain-language guidance using fixed thresholds - for example flagging a scaled calorie value at or above 700 kcal as “a large calorie load for one sitting” and a sodium value above 1200 mg as “very high” - so the numeric result

of equations (2)-(3) is always paired with an actionable sentence rather than a bare percentage.

B. Health-Aware Default Ordering

Wherever the UI needs to suggest foods without an explicit query (for example the default options in a food picker), get_food_choices() ranks candidates with a hand-weighted health score that rewards protein and fiber and penalizes fat, sugar and sodium:

$$score = 1.2 \cdot prot + 0.8 \cdot fib - 0.5 \cdot fat - 0.2 \cdot sug - Na/1000 \quad (4)$$

where prot, fib, fat and sug are the food's protein, fiber, fat and sugar in grams and Na is sodium in milligrams. Foods are then sorted by this score, ties broken by protein descending and calories ascending, and the deduplicated, non-placeholder top results (default limit 120) are offered as choices - a lightweight substitute for a learned ranking model that keeps the ordering fully explainable from four coefficients.

C. Condition-Based Recommendations

Health Recommendations and the chat assistant's condition-fit intent (Section VI) share one verdict table (Table II): each of five conditions maps to a boolean expression over the dataset's dietary flags and numeric nutrients. A food "fits" a condition only when its boolean expression evaluates true, which keeps every verdict traceable to a single readable rule rather than a model score.

Condition	Fit rule
Diabetes	diabetic_friendly AND sugar ≤ 10 g
Hypertension	heart_healthy AND sodium ≤ 400 mg
Heart disease	heart_healthy AND fat ≤ 15 g AND sodium ≤ 400 mg
Weight loss	weight_loss_friendly AND calories ≤ 220 kcal
Weight gain	calories ≥ 220 kcal AND protein ≥ 15 g

Safety note: These condition-fit rules are project-level heuristics, not clinically validated dietary recommendations. Users with medical conditions or severe food allergies should verify food choices with a qualified healthcare professional.

TABLE II. CONDITION-FIT RULES SHARED BY HEALTH RECOMMENDATIONS AND THE CHAT ASSISTANT.

D. Recipe Generation

generate_recipe() first returns a dataset-authored recipe when one exists; otherwise it synthesizes one from the food's ingredient list. Ingredient tokens are split on commas, a cooking style (simmer, roast, mix, or the stir-fry default) is chosen from keyword matches on the food's name ("curry"/"bhuna"/"korma" → simmer; "roast"/"grilled"/"tandoori" → roast; "salad"/"chutney"/"raita" → mix), and an estimated cooking time in minutes is computed from the ingredient count and calorie density:

$$time = clip(8 + 2n + calories/80, 12, 40) \quad (5)$$

where n is the number of ingredient tokens. A companion function, estimate_ingredient_amounts(), spreads a target cooked weight across the ingredient list by classifying each token as a seasoning, fat, liquid, or bulk ingredient against fixed lookup sets, assigning small fixed per-100 g amounts to the first three classes, and dividing the remaining weight across bulk ingredients with the first bulk ingredient weighted twice any other - a simple, explainable stand-in for a true recipe-scaling model that nonetheless produces plausible gram and teaspoon/tablespoon amounts for any of the 1,254 dishes.

As a worked example, generate_recipe() applied to "Rui Bhuna" (ingredients: Rui, oil, spices) has no stored recipe text, so it falls into the synthesis path: three ingredient tokens give n = 3, and with calories = 218.6 kcal, equation (5) evaluates to clip(8 + 6 + 2.73, 12, 40) ≈ 16.7 minutes; the food's name contains "Bhuna", which is not itself a style keyword, so the style defaults to stir-fry (rather than the arguably more accurate simmer, since "bhuna" is a semi-dry frying technique the current keyword list does not recognize - a concrete instance of the coverage limitation discussed in Section VIII). estimate_ingredient_amounts() for a 250 g target classifies "spices" as a seasoning (fixed 5.0 g) and "oil" as a fat (fixed 15.0 g), leaving 230 g distributed across the single bulk ingredient, "Rui", which therefore receives the full remainder.

E. Calorie-Targeted Meal Planning

build_meal_plan() splits a daily calorie target across four meals using fixed shares - Breakfast 25%, Lunch 35%, Dinner 30%, Snack 10% - selects, from a condition-filtered pool and a per-meal calorie band, the single candidate with the highest protein for each meal, and then solves for the serving weight that lands closest to that meal's calorie share:

$$grams = clip(g0 \cdot (mt / k0), 0.4 \cdot g0, 3.0 \cdot g0) \quad (6)$$

where g0 and k0 are the candidate food's base serving weight and calories and mt is that meal's share of the daily calorie target, clipping the result between 0.4x and 3.0x the dataset's own base serving so that the solved portion never becomes implausibly small or large. The four scaled meals and their nutrient totals are returned together, giving both the Meal Planner page and the chat assistant's meal-plan intent (Section VI) a full day's plan from one function call.

F. Menu and Free-Text Matching

The Menu Analyzer accepts one food per line of free text and resolves each line to a dataset row with get_best_match(): an exact, case-insensitive match is tried first, and failing that, difflib.get_close_matches() ranks candidates by Ratcliff/Obershelp string similarity (cutoff 0.45), returning up to five suggestions when no match clears the threshold. This same primitive underlies the Chat Assistant's calorie, recipe and condition-fit intents (Section VI), so the two features never disagree about what counts as "close enough" to a dataset name.

G. Diet and Allergen Filtering

The Diet & Allergen Filter tool applies apply_food_filters() across the full dataset using the rules of Section III-B, and, per Section III-B, surfaces the specific reason each excluded food failed rather than a bare pass/fail count, which the project's evaluation (Section VII) found made incorrect exclusions far easier to spot during manual testing than a silent filter would have.

H. Side-by-Side Comparison

Compare Foods pits any two dataset foods against each other across the seven tracked nutrients. For calories, fat, sugar and sodium the lower value wins; for protein and fiber the higher value wins; carbohydrates are treated as neutral since neither direction is universally healthier. Each of the six directional nutrients contributes one point to whichever food wins it, and the food with more points is declared the overall winner, with ties reported explicitly rather than arbitrarily broken.

VI. CONVERSATIONAL ASSISTANT PIPELINE

The Chat Assistant (chatbot.py) is the one part of FoodGPT that can call an external large language model, and it is built as a two-stage, retrieval-first pipeline (Fig. 2) specifically to minimize how often - and how far - it relies on the model's own claims about nutrition.

A. Stage 1: Rule-Based Intents

Every incoming message is tested, in order, against five regular-expression intents: a calorie lookup ("how many calories in X"), a condition-fit question ("is X good for Y"), a recipe request ("recipe for X" / "how do I cook X"), a meal-plan request ("meal plan for N calories"), and a trait list ("what foods are vegan"). The first intent that both matches the message's phrasing and resolves to a real dataset row (via get_best_match(), Section V-F) answers the message directly from the cleaned DataFrame - deterministic, instant, and, because no generative model is involved, impossible to hallucinate.

B. Stage 2: Retrieval-Grounded LLM Fallback

If no rule matches, retrieve_context_rows() tokenizes the message and scores every dataset row by how many tokens overlap its food name, category, cuisine and tags, keeping the top six rows as context. These rows are formatted into a short block of bullet facts and appended to a system prompt that explicitly instructs the model to treat that block as its only source of truth and to say so plainly if nothing relevant was retrieved, rather than inventing a dataset entry. The message, this grounding block, and the recent chat history are then sent to Groq's openai/gpt-oss-20b model. If the call succeeds, its reply is returned as-is; if the Groq API key is simply not configured, the assistant says so and continues to answer from the dataset alone; and if the call fails for any other reason (network error, decommissioned model, rate limit), the assistant falls back to a short, dataset-only reply built from the same retrieved rows plus a plain-language note, so a Groq outage degrades the assistant's fluency but never crashes it or causes it to fabricate an answer.

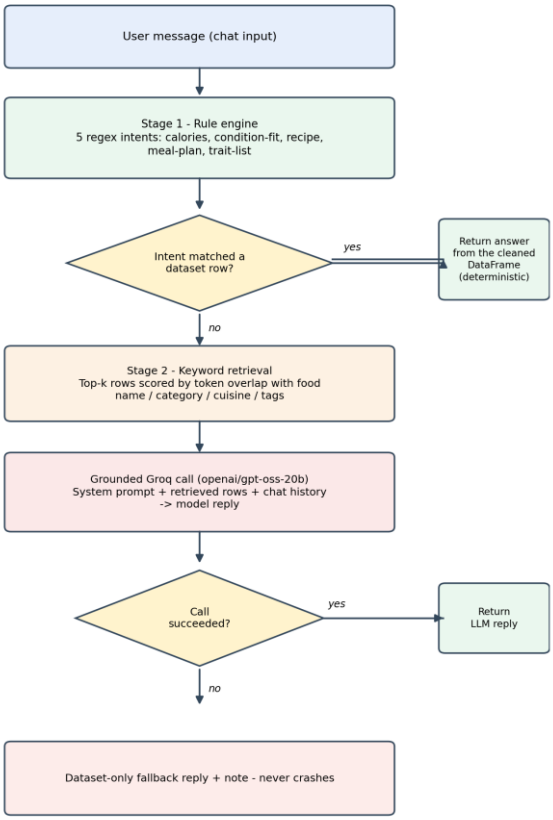

Fig. 2. Two-stage chat pipeline: deterministic rule matching first, then a retrieval-grounded, fail-soft LLM fallback.

VII. IMPLEMENTATION

A. Tech Stack

FoodGPT is implemented in Python. Streamlit [9] provides both the UI framework and the multi-page router (st.navigation / st.Page, wired from main.py), which was chosen over a full frontend/backend split because it lets a small team ship nine data-driven pages without writing any JavaScript, at the cost of coarser control over layout than a bespoke web frontend would offer. pandas performs all dataset loading, cleaning and querying; every core function in Sections III and V operates on a pandas DataFrame or a row Series. The Groq Python client [10] is the sole third-party dependency introduced for Stage 2 of the chat pipeline (Section VI-B); it is imported lazily inside call_groq() so that the other 2,000+ lines of the codebase have no import-time dependency on it and, by extension, no import-time requirement for an API key. Application-wide styling, the dark colour theme, and the shared top navigation bar live in theme.py and are applied by a single apply_theme() call at the top of every page, which keeps the nine pages visually consistent without a shared layout template engine. requirements.txt lists three top-level dependencies (streamlit, pandas, groq); no database, message queue or containerized backend is used, since the entire application state is either the cached DataFrame or ordinary Streamlit session state.

The UI itself is deliberately minimal in its dependencies: theme.py injects one CSS block through st.markdown() to set a dark colour palette, card and pill components, and the top navigation bar, rather than pulling in a component library, so the entire visual language of all nine pages is defined in roughly 300 lines that any contributor can read end to end. Interactive state - the two foods selected in Compare Foods, the menu text typed into Menu Analyzer, the running conversation in Chat Assistant - is held in ordinary Streamlit session state and is intentionally not persisted between sessions, which keeps the application stateless from a deployment perspective (any server restart or redeploy loses no data because none was meant to survive one) at the cost of the personalization features listed as future work in Section IX.

B. Reproducibility

The project has no external service dependency beyond the optional Groq call: the entire application runs from python -m venv, pip install -r requirements.txt, and streamlit run main.py, after which every one of the nine pages is reachable from the dashboard hero or the top navigation bar without any database migration or seed step, since the only persistent state is the checked-in CSV. The codebase currently has no automated unit-test suite; correctness in this report (Section VIII) was instead established by direct, manual comparison of tool output against the raw dataset and against the source functions listed in Sections III, V and VI, which the authors consider adequate for a dataset-driven, side-effect-free application of this size but flag as the clearest process improvement for a larger version of the project.

VIII. EVALUATION AND DISCUSSION

Because seven of the eight tools are deterministic functions of the cleaned dataset, they were evaluated primarily by manual, exploratory testing rather than a held-out test set: for each tool, a range of known foods (common Bangladeshi dishes such as Rui Bhuna and Khichuri, and international dishes present in the dataset) were queried and their outputs checked against the raw CSV by hand, and the allergen and diet-filter reason strings (Section III-A, V-G) were spot-checked against each food's listed ingredients to confirm that "contains" and "may contain" labels matched a human reading of the ingredient text. The Chat Assistant's rule-based stage was exercised with paraphrases

of its five supported intents to confirm that reasonable phrasing variants still matched, and its LLM fallback was tested both with and without a configured Groq API key to confirm the two degraded-mode paths described in Section VI-B behave as designed.

Tool / query	Observed result
Nutrition Analysis: calories in Rui Bhuna	218.6 kcal, 17.7 g protein, 4.7 g fat per 100 g listed serving
Allergen estimate: Rui Bhuna	"Contains Fish" (from "Rui" in the name/ingredients)
Condition fit: Rui Bhuna for diabetes	Fits - diabetic_friendly = Yes and sugar 1.1 g ≤ 10 g
Compare: Rui Bhuna vs. Rui Jhol (Curry)	Rui Bhuna wins 3-2 (protein, fat, sugar) vs. (calories, sodium); fiber tied
Meal Planner: 1,800 kcal, diabetes	Breakfast: Mutton Grilled, 424 g, 450 kcal; Lunch: Duck BBQ, 338 g, 630 kcal; Dinner: Duck BBQ, 290 g, 540 kcal; Snack: Plain Dal, 150 g, 180 kcal; totals 1,800 kcal, 173.1 g protein

TABLE III. ILLUSTRATIVE OUTPUTS RECORDED DURING MANUAL TESTING (ACTUAL DATASET VALUES).

The meal-plan example in Table III also surfaced a genuine limitation, not just a demonstration of correctness: because build_meal_plan() (Section V-E) chooses the highest-protein candidate independently for each meal without excluding foods already used earlier in the same day, and does not filter candidates by the dataset's own meal_type column, the planner reused Duck BBQ for both lunch and dinner and selected a grilled-mutton dish for breakfast - numerically correct against the 1,800 kcal target and the diabetes condition rule, but not a plan a person would actually cook in one day. This is recorded here as an example of the kind of plausibility gap that unit-testing the arithmetic alone would not have caught, and is the basis for the meal-repetition and meal-type filtering improvement listed in Section IX.

Threats to validity should be interpreted alongside the absence of clinical validation, the use of heuristic allergen estimation, and the manual rather than benchmark-based evaluation.

The dominant limitation of this design is coverage, not correctness: because every tool ultimately answers from one 1,254-row dataset, a food that is absent, misspelled beyond the fuzzy-matching threshold, or a regional dish not represented at all, cannot be served by the seven deterministic tools and can only be answered - if at all - by the LLM fallback's general knowledge, which is exactly the path this project otherwise tries to avoid relying on for nutrition facts. The keyword-rule allergen estimator (Section III-A) is similarly precision-oriented rather than exhaustive: it will under-flag an allergen expressed through an ingredient word not present in its keyword lists, and users with severe allergies are cautioned in the UI copy that allergen labels are estimated, not verified. Finally, the condition-fit rules of Table II are deliberately simple boolean thresholds rather than a personalized or clinically validated model, and are presented as general guidance rather than medical advice.

A further threat to validity is representativeness: the manual test cases described above were drawn from foods the authors already recognized, which biases testing toward well-formed, common dataset entries and away from the sparse, oddly named, or duplicate-looking rows that a larger, crowd-sourced nutrition dataset inevitably contains. Because 906 of 1,254 rows are labeled purely Bangladeshi and the remainder cover a long tail of international and fusion cuisines with far fewer examples each, tools such as Health Recommendations and Meal Planner are, in practice, better tested and better populated for Bangladeshi dishes than for any single international cuisine, a bias that is inherited directly from the source dataset rather than introduced by FoodGPT's own logic, but that future evaluation should measure explicitly (for example, hit-rate on Menu Analyzer broken out by cuisine label) rather than assume away.

These trade-offs were made deliberately for a project of this scope: a transparent, rule-based core is easier to verify, easier to debug when a specific recommendation looks wrong, and safer to reason about for allergy- and condition-sensitive output than an opaque learned model would be, while confining the one genuinely open-ended component - free-form conversation - to a model that is grounded in, and can fall back to, the same trusted dataset.

IX. CONCLUSION AND FUTURE WORK

FoodGPT demonstrates that a single cleaned dataset, shared through a small set of well-factored core modules, can support eight distinct, mutually consistent nutrition tools plus a conversational assistant, with a large language model used only where a deterministic rule genuinely cannot answer the question and always grounded in retrieved data when it is used. Future work includes: (1) constraining build_meal_plan() to avoid repeating a food across meals in the same day and to respect the dataset's own meal_type column, directly addressing the plausibility gap surfaced in Table III; (2) expanding the dataset's regional and international coverage to reduce the fuzzy-matching and LLM-fallback burden described in Section VIII; (3) replacing the hand-tuned health-score coefficients of Section V-B with weights learned from user feedback once real usage data exists; (4) extending the keyword-based allergen estimator with a small, human-reviewed named-entity model for higher recall on rare allergen phrasings; and (5) adding a lightweight per-user profile (persisted allergies, conditions, and calorie targets) so that the Meal Planner and Chat Assistant no longer require the same inputs to be re-entered in every session.

Beyond the specific tools, the broader lesson from building FoodGPT is that a large language model is most useful, and safest, at the edges of an application rather than at its center. Placing the LLM behind a rule-based first stage and a retrieval-grounded second stage (Section VI) meant that the vast majority of everyday questions - a calorie lookup, a condition check, a recipe request - never touched the model at all, and the minority that did were still anchored to the same trusted rows every other tool reads. For a course project with a hard deadline and no budget for human evaluation of model outputs at scale, that architecture did more to make the final system trustworthy than any amount of prompt engineering on the LLM call itself would have.

REFERENCES

[1] R. Y. Toledo, A. A. Alzahrani, and L. Martínez, "A Food Recommender System Considering Nutritional Information and User Preferences," IEEE Access, vol. 7, pp. 96695–96711, 2019, doi: 10.1109/ACCESS.2019.2929413.

[2] R. Y. Toledo and L. Martínez, "A Health-Awareness Nutrition Recommender System," in Proc. IEEE 14th Int. Conf. on Intelligent Systems and Knowledge Engineering (ISKE), 2019, pp. 36–42, doi: 10.1109/ISKE47853.2019.9170432.

[3] A. B. Ocay, J. M. Fernandez, and T. D. Palaoag, "NutriTrack: Android-based food recognition app for nutrition awareness," in Proc. 2017 3rd IEEE Int. Conf. on Computer and Communications (ICCC), 2017, doi: 10.1109/CompComm.2017.8322907.

[4] M. Han, J. Chen, and Z. Zhou, "NutrifyAI: An AI-Powered System for Real-Time Food Detection, Nutritional Analysis, and Personalized Meal Recommendations," arXiv preprint arXiv:2408.10532, 2024.

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela,

"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020.

[6] W. Xiong, B. Zeng, J. Wang, M. Hao, and H. Che, "Research Progress in Food Allergen Labeling Management and Its Enlightenment to China," Food Science, vol. 44, no. 15, pp. 329–350, 2023, doi: 10.7506/spkx1002-6630-20220625-291.

[7] W. Xiong, C. H. Parker, C. C. Boo, and K. L. Fiedler, "Comparison of allergen quantification strategies for egg, milk, and peanut in food using targeted LC-MS/MS," Analytical and Bioanalytical Chemistry, vol. 413, no. 23, pp. 5755–5766, 2021, doi: 10.1007/s00216-021-03550-x.

[8] A. G. M. Mactal, C. K. T. Honrada, B. M. Zapata, J. M. A. Silva, A. D. David, and L. G. V. Villanueva, "AlertGen: Advanced Allergen Detection for Labelled and Non-labelled Foods," Asian Journal of Multidisciplinary Studies, vol. 9, no. 1, pp. 58–70, 2026.

[9] Streamlit, "Streamlit documentation," Streamlit, 2026. [Online]. Available: https://docs.streamlit.io/

[10] Groq, "Groq API documentation and Python SDK," Groq, 2026. [Online]. Available: https://console.groq.com/docs/quickstart

[11] Dataset. Available: https://github.com/tahsinramisha/FoodGpt/blob/main/data/bd_food_nutrition_dataset.csv